第6章　树和二叉树 作业（共11题）

6.5　6.8　6.17　6.25　6.26　6.29　6.42　6.43　6.47　6.49
6.65

第6章　树和二叉树
（Tree & Binary Tree）

6.1　树的基本概念

6.2　二叉树

6.3　遍历二叉树和线索二叉树

6.4　树和森林

6.5　赫夫曼树及其应用

二叉树的定义、性质和存储结构

二叉树的运算

6.2 二叉树

为何要重点研究每结点最多只有两个 “叉” 的树?

- ✓ **二叉树的结构最简单，规律性最强；**
- ✓ **可以证明，所有树都能转为唯一对应的二叉树，不失一般性。**

6.2.1 二叉树的定义

6.2.2 二叉树的性质

6.2.3 二叉树的存储结构

注：二叉树最重要的运算是：遍历！

6.2.2 二叉树的性质 (3+2)

性质1：**在二叉树的第i层上至多有2^{i-1}个结点（i>0）**

性质2：**深度为k的二叉树至多有2^k-1个结点（k>0）**

性质3：**对于任何一棵二叉树，若2度的结点数有n_2个，则叶子数（n_0）必定为n_2+1 （即$n_0=n_2+1$）**

性质4：**具有n个结点的完全二叉树的深度必为$\lfloor \log_2 n \rfloor + 1$**

性质5：**对完全二叉树，若从上至下、从左至右编号，则编号为i的结点，其左孩子编号必为$2i$，其右孩子编号为$2i+1$；其双亲的编号必为$i/2$（$i=1$时为根,除外）。**

课堂练习：

1. 树 T 中各结点的度的最大值称为树 T 的 D 。

A) 高度　B) 层次　C) 深度　D) 度

2.深度为 K 的二叉树的结点总数，最多为 C 个。

A) 2^{k-1}　B) $\log_2 k$　C) $2^k - 1$　D) 2^k

3. 深度为9的二叉树中至少有 C 个结点。

A) 2^9　B) 2^8　C) 9　D) $2^9 - 1$

讨论：二叉树的叶子数和度为2的结点数之间有关系吗？

性质3：**对于任何一棵二叉树，若2度的结点数有n_2个，则叶子数（n_0）必定为n_2+1 （即$n_0=n_2+1$）**

物理意义：叶子数 = 2度结点数 + 1

证明：

∵ 二叉树中全部结点数$n=n_0+n_1+n_2$(叶子数 + 1度结点数 + 2度结点数)

又∵二叉树中全部结点数$n=B+1$（总分支数 + 根结点）

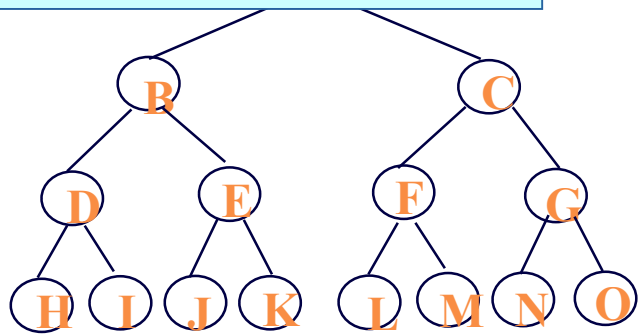
（除根结点外，每个结点必有一个直接前趋，即一个分支）

而 总分支数$B=n_1+2n_2$ (1度结点必有1个直接后继，2度结点必有2个)

三式联立可得： $n_0+n_1+n_2=n_1+2n_2+1$，即$n_0=n_2+1$

解释：完全二叉树的特点是只有最后一层叶子不满，且全部集中在左边。但这其实是顺序二叉树的含义。而图论中的“完全二叉树”是指$n_1=0$的情况。

完全二叉树：深度为k的、有n个结点的二叉树，当且仅当其每一个结点都与深度为k的满二叉树中编号从1至n的结点一一对应。



深度为4的满二叉树

A

满二叉树和完全二叉树有什么区别?

答：满二叉树是叶子一个也不少的树，而完全二叉树虽然前k-1层是满的，但最底层却允许在右边缺少连续若干个结点。满二叉树是完全二叉树的一个特例。

对于两种特殊形式的二叉树（满二叉树和完全二叉树），还特别具备以下2个性质：

性质4： **具有n个结点的完全二叉树的深度必为$\lfloor \log_2 n \rfloor + 1$**

证明：根据性质2，深度为k的二叉树最多只有2^k-1个结点，且完全二叉树的定义是与同深度的满二叉树前面编号相同，即它的总结点数n位于k层和k-1层满二叉树容量之间，
即 $2^{k-1}-1<n\le 2^k-1$ 或$2^{k-1}\le n<2^k$
三边同时取对数，于是有：$k-1\le \log_2 n<k$ 因为k是整数，所以$k=\lfloor \log_2 n \rfloor +1$

性质5： **对完全二叉树，若从上至下、从左至右编号，则编号为i的结点，其左孩子编号必为$2i$，其右孩子编号为$2i+1$；其双亲的编号必为$i/2$（$i=1$时为根,除外）。**

可根据归纳法证明。

例：

一棵完全二叉树有1000个结点，则它必有__~~499~~__个叶子结点，有__~~498~~__个度为2的结点，有__1__个结点只有非空左子树，有__0__个结点只有非空右子树。

分析题意：已知n=1000,求n_0和n_2,还要判断末叶子是挂在左边还是右边?

请注意：叶子结点总数≠末层叶子数！！！

正确答案：

全部叶子数=⌈1000/2⌉=500个。

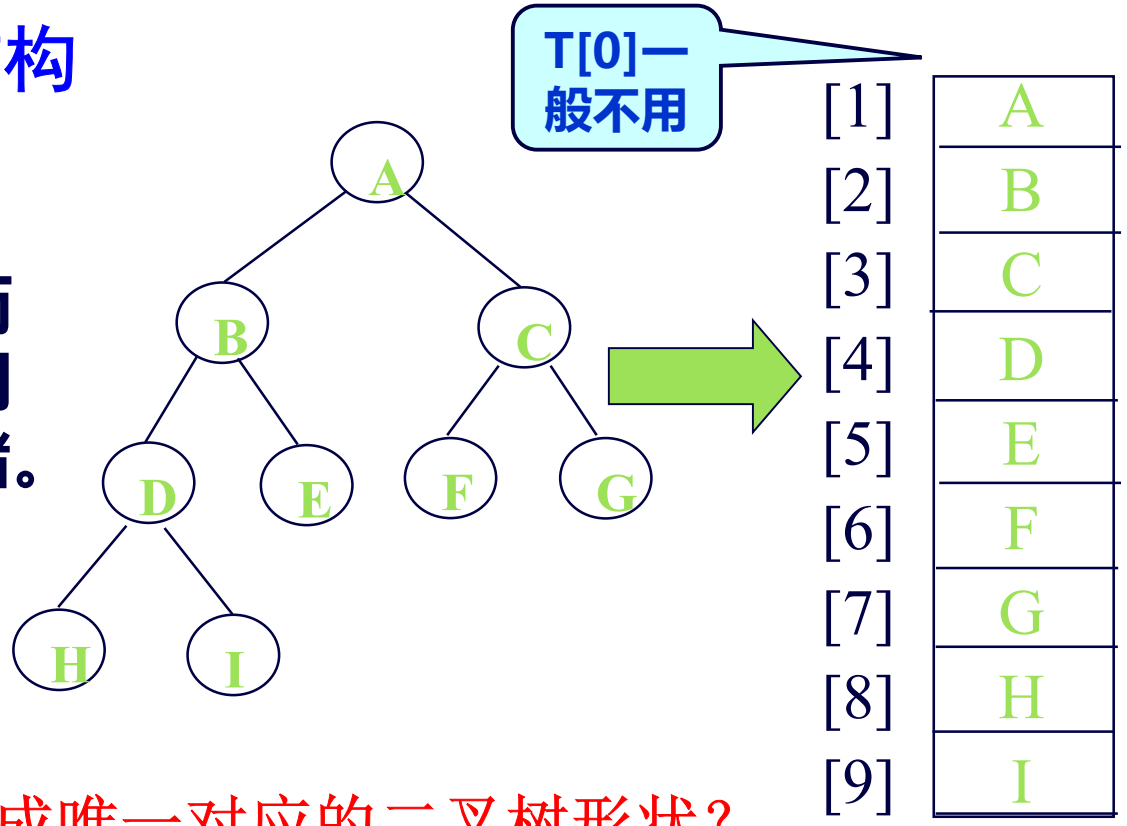
度为2的结点=叶子总数 - 1=499个。

因为最后一个结点坐标是偶数，所以必为左子树。

6.2.3 二叉树的存储结构

一、顺序存储结构

按二叉树的结点“自上而下、从左至右”编号，用一组连续的存储单元存储。



问：顺序存储后能否复原成唯一对应的二叉树形状？

答：若是完全/满二叉树则可以做到唯一复原。

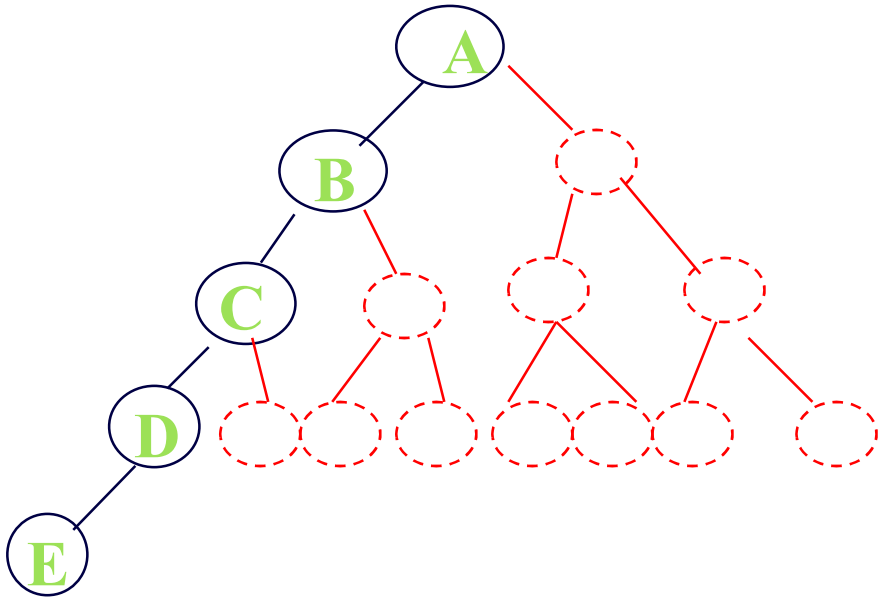
而且有规律：下标值为i的双亲，其左孩子的下标值必为2i，其右孩子的下标值必为2i＋1（即性质5）

例如，对应[2]的两个孩子必为[4]和[5],即B的左孩子必是D,右孩子必为E。

讨论：不是完全二叉树怎么办?

答：一律转为完全二叉树！

方法很简单，将各层空缺处统统补上“虚结点”，其内容为空。



[1]	A
[2]	B
[3]	^
[4]	C
[5]	^
[6]	^
[7]	^
[8]	D
[9]	^
.	…
[16]	E

缺点：①浪费空间；②插入、删除不便

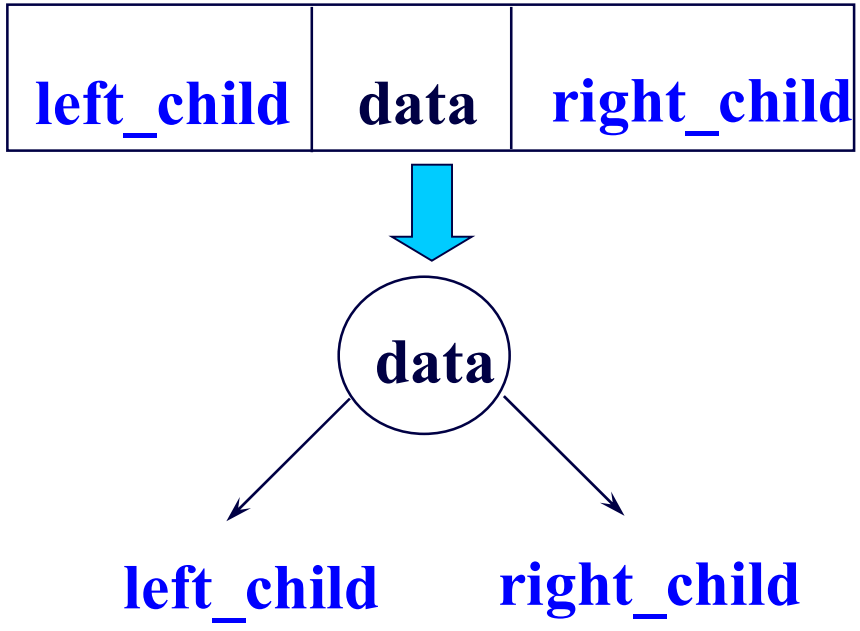
二、链式存储结构

用二叉链表即可方便表示。

一般从根结点开始存储。

（相应地，访问树中结点时也只能从根开始）

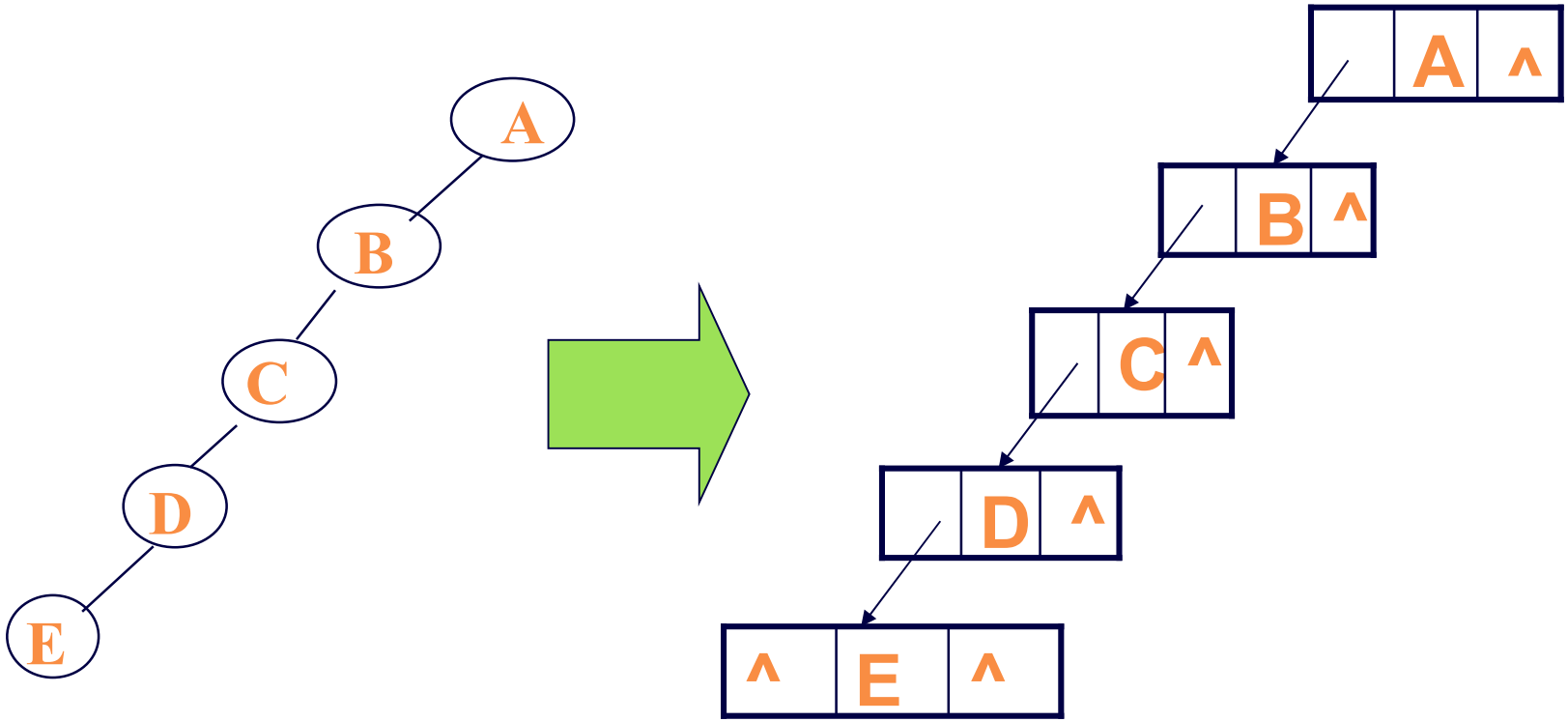
注：如果需要倒查某结点的双亲，可以再增加一个双亲域（直接前趋）指针，将二叉链表变成三叉链表。



二叉树结点数据类型定义：

```
typedef struct node *tree_pointer;
typedef struct node{
 int data;
 tree_pointer  left_child,  right_child;
} node;
```

二叉树链式存储举例：



优点：①不浪费空间；②插入、删除方便

6.3 遍历二叉树和线索二叉树

6.3.1 遍历二叉树

Traversing Binary Tree

遍历定义—— **指按某条搜索路线遍访每个结点且不重复（又称周游）。**

遍历用途—— **它是树结构插入、删除、修改、查找和排序运算的前提，是二叉树一切运算的基础和核心。**

遍历方法—— **对每个结点的查看通常都是“先左后右”。**

遍历规则———

- 二叉树由根、左子树、右子树构成，定义为D、 L、R
- D、 L、R的组合定义了六种可能的遍历方案：

  LDR，LRD，DLR，DRL，RDL，RLD

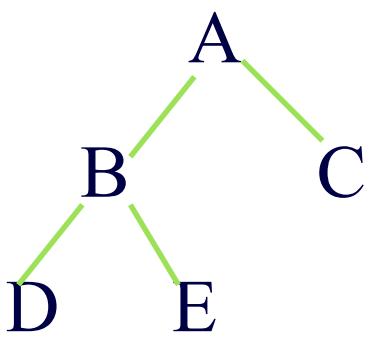
- 若限定先左后右，则有三种实现方案：

DLR	LDR	LRD
先序遍历	中序遍历	后序遍历

注：“先、中、后”的意思是指访问的结点D是先于子树出现还是后于子树出现。

以根结点为参照系

例1：



先序遍历的结果是：A B D E C

中序遍历的结果是：D B E A C

后序遍历的结果是：D E B C A

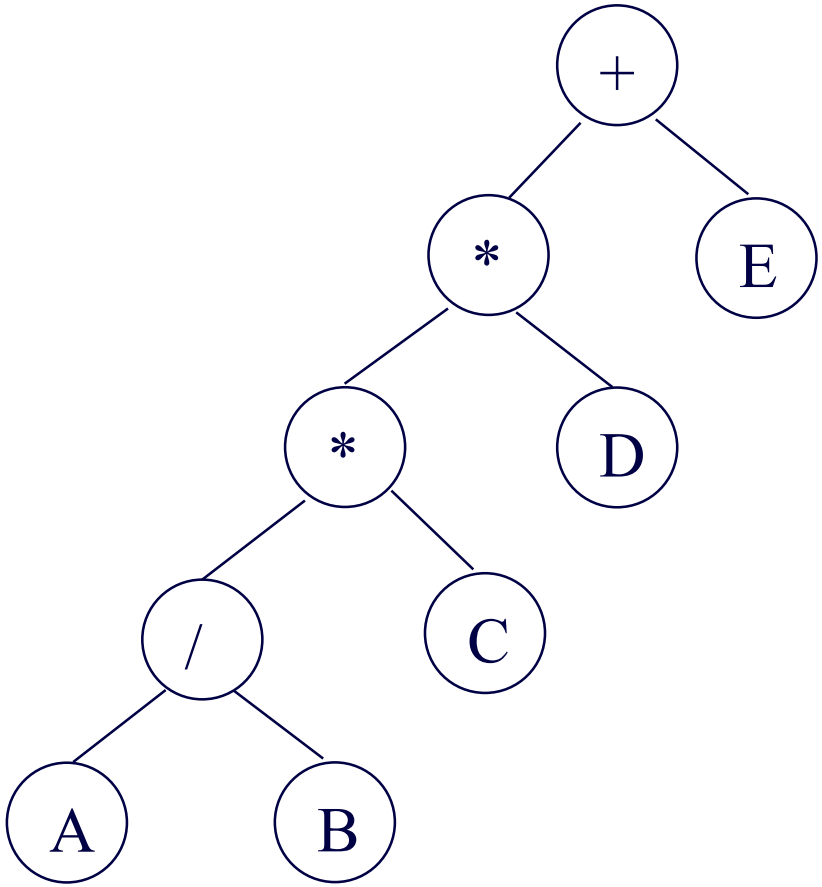
口诀：

DLR—先序遍历，即先根再左再右

LDR—中序遍历，即先左再根再右

LRD—后序遍历，即先左再右再根

例2：用二叉树表示算术表达式



先序遍历结果

+ * * / A B C D E

—前缀表示法

中序遍历结果

A / B * C * D + E

—中缀表示法

后序遍历结果

A B / C * D * E +

—后缀表示法

层次遍历结果

+ * E * D / C A B

结点数据类型自定义

```
typedef struct node{
int data;
struct node *lchild,*rchild ;
} node;
node *root;
```

先序遍历算法

```
DLR( node *root )
{if (root !=NULL) //非空二叉树
 {printf("%d",root->data); //访问D
DLR(root->lchild); //递归遍历左子树
DLR(root->rchild); //递归遍历右子树
  } return(0); }
```

中序遍历算法

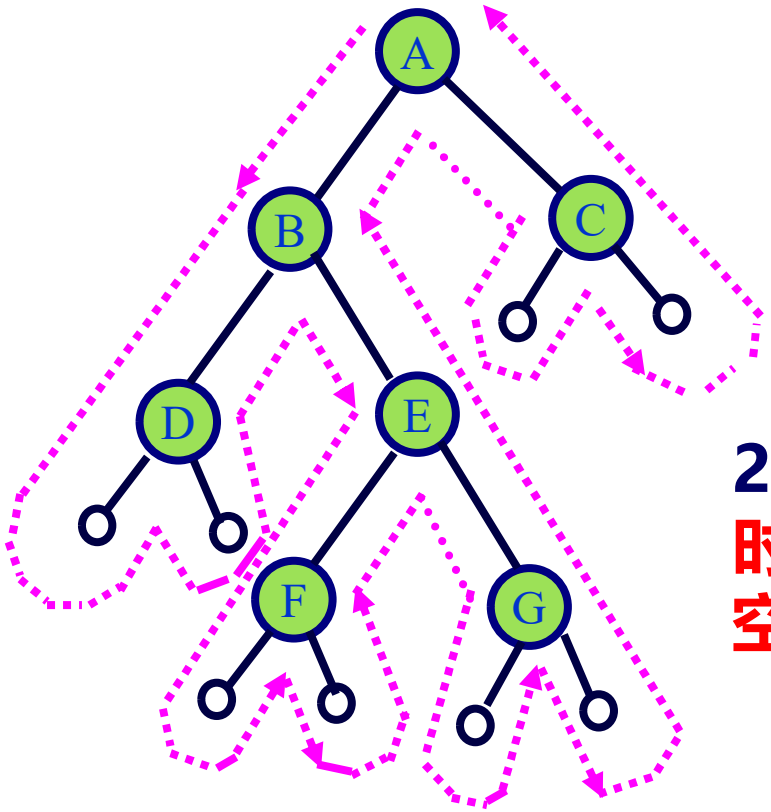
```
LDR(node *root)
{if(root !=NULL)
  {LDR(root->lchild);
  printf("%d",root->data);
  LDR(root->rchild);
   } return(0);}
```

后序遍历算法

```
LRD (node *root)
{if(root !=NULL)
{LRD(root->lchild);
  LRD(root->rchild);
  printf("%d",root->data);
} return(0);}
```

对遍历的分析：

1. 从前面的三种遍历算法可以知道：如果将print语句抹去，从递归的角度看，这三种算法是完全相同的，或者说这三种遍历算法的访问路径是相同的，只是访问结点的时机不同。



从虚线的出发点到终点的路径上，每个结点经过3次。

第1次经过时访问，是先序遍历
第2次经过时访问，是中序遍历
第3次经过时访问，是后序遍历

2. 二叉树遍历的时间效率和空间效率

时间效率:O(n) //每个结点只访问一次
空间效率:O(n) //栈占用的最大辅助空间

精确值：树深为k的递归遍历需要k+1个辅助单元

例：【严题集6.42③】编写递归算法，计算二叉树中叶子结点的数目。

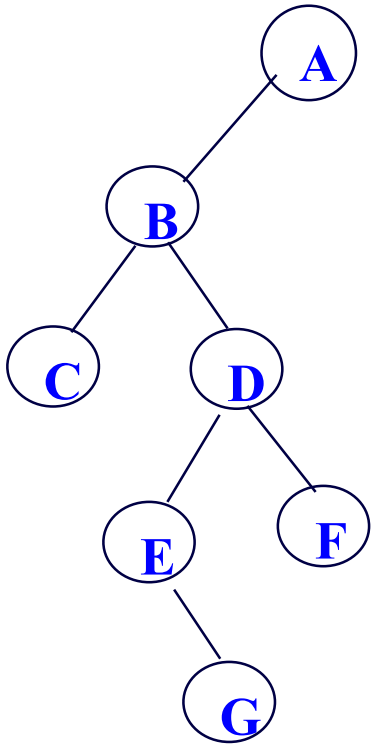
思路：若左右指针均空，必为叶子。可选用任何一种遍历算法查找叶子，将其统计并打印出来。

```
DLR(node *root)     //采用先序遍历的递归算法
{ if ( root!=NULL )  //非空二叉树条件，等效于 if(root)
  { if(!root->lchild&&!root->rchild)  //是叶子结点则统计并打印
        {sum++; printf("%d\n",root->data);}
    DLR(root->lchild);     //递归遍历左子树，直到叶子处;
    DLR(root->rchild);    //递归遍历右子树，直到叶子处;
  }
return(0);  }
```

如何**把二叉树存入电脑内?**

怎样建树？见教材P131例

例：将下面的二叉树以二叉链表形式存入计算机内。



考虑1：输入结点时怎样表示“无孩子”？

考虑2：以何种遍历方式来输入和建树?

字符串输完后应当再加一特殊的结束符号(如$)，因为Φ Φ Φ无法惟一表示结束。

用空格字符表示‘无孩子’或指针为空

将二叉树按先序遍历次序输入：

A B C Φ Φ D E Φ G Φ Φ F Φ Φ Φ (/n)

输入序列： A B C Φ Φ D E Φ G Φ Φ F Φ Φ Φ

建树算法：

```
Status CreateBiTree( BiTree &T ){    //构造二叉树T
scanf("%c",&ch);
if(ch==“”)T=NULL;
else{
    if(!(T=( BiTNode*)malloc(sizeof(BiTNode))))exit(overflow);
    T->data=ch;                          //生成根结点
    CreateBiTree(T->lchild);     //构造左子树
    CreateBiTree(T->rchild);   //构造右子树
} return OK;
} //CreateBiTree
```

特别讨论：若已知先序（或后序）遍历结果和中序遍历结果，能否“恢复”出二叉树?

【严题集6.31④】请证明：由一棵二叉树的先序序列和中序序列可唯一确定这棵二叉树。

例：已知一棵二叉树的中序序列和后序序列分别是BDCEAFHG和 DECBHGFA，请画出这棵二叉树。

分析：

①由后序遍历特征，根结点必在后序序列尾部（即A）；

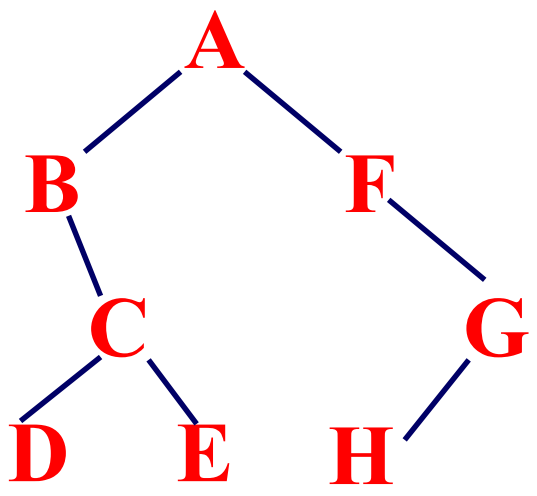
②由中序遍历特征，根结点必在其中间，而且其左部必全部是左子树的子孙（即BDCE），其右部必全部是右子树的子孙（即FHG）；

③继而，根据后序中的DECB子树可确定B为A的左孩子，根据HGF子串可确定F为A的右孩子；以此类推。

详细说明：

已知中序遍历：B D C E A F H G

已知后序遍历：D E C B H G F A



（D C E）　　（F H G）

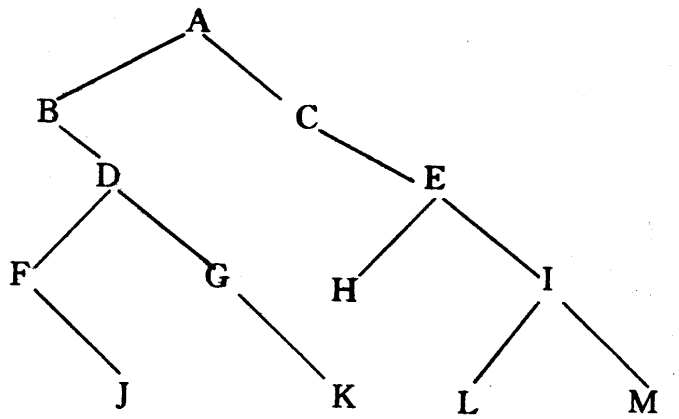
04级期末考题： 由1001个结点构成的二叉树，何种型态叶子结点最多，何种叶子最少？分别给出叶子结点个数和度为1结点的个数。

答： 因为 n=n0+n1+n2

完全二叉树叶子最多----叶子结点501个,度为1结点的个数为0；

单支树的叶子最少----叶子结点个数为1，度为1结点的个数1000

夜大考题： 试写出如图所示的二叉树按中序遍历时得到的结点序列。



答： LDR：B F J D G K A C H E L I M

6.3.2 线索二叉树（仅介绍，不考）

Threaded Binary Tree

讨论：用二叉链表法（l_child, r_child）存储包含n个结点的二叉树，结点的指针区域中会有多少个空指针？

有n+1个！

证明：**用二叉链表存储包含n个结点的二叉树，结点必有2n个链域（见二叉链表数据类型说明）。**

除根结点外，二叉树中每一个结点有且仅有一个双亲，意即每个结点地址占用了双亲的一个直接后继，n个结点地址共占用了n-1个双亲的指针域。也就是说，只会有n - 1个结点的链域存放指针。

所以，空指针数目＝2n－(n-1)=n+1个。

结论：用二叉链表法存储包含n个结点的二叉树，结点的指针区域中会有n+1个空指针。

可以用它来存放当前结点的直接前驱和后继等线索，以加快查找速度。这就是线索二叉树的意义和用途。

疑问1：二叉树是1:2的非线性结构，如何定义其惟一的直接后继?
答：要遍历之后才能得到，且不同遍历算法得到的后继也不同。先依遍历规则把每个结点对应的前驱或后继线索预存起来，这叫做“线索化”。

疑问2：获得这种“直接前驱”或“直接后继”有何意义?
答：从任一结点出发都能快速找到其前驱和后继，且不必借助堆栈

疑问3：如何经济的(预先)存放这类信息?
答：左孩子/前驱复用，右孩子/后继复用，后者称之为线索

疑问4：计算机如何识别是孩子指针还是线索指针?

为识别复用的两种不同信息，特增加两个标志域：

lchild	LTag	data	RTag	rchild

各1bit

约定：

当Tag域为0时,表示正常情况;

当Tag域为1时,表示线索情况.

左(右)孩子

前驱(后继)

问：增加了前驱和后继等线索有什么好处?

——能方便找出当前结点的前驱和后继，不用堆栈（递归）也能遍历整个树。

1. 有关线索二叉树的几个术语：

线索链表：用含Tag的结点样式所构成的二叉链表

线　索：指向结点前驱和后继的指针

线索二叉树：加上线索的二叉树

线　索　化：对二叉树以某种次序遍历使其变为线索二叉树的过程

线索化过程就是在遍历过程中修改空指针的过程：

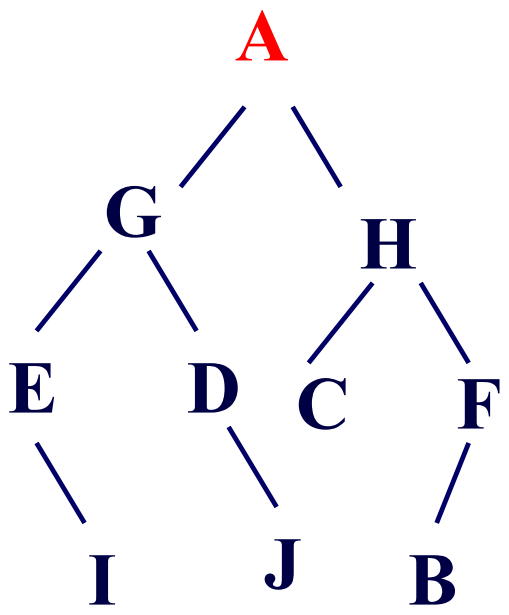
将空的lchild改为结点的直接前驱；

将空的rchild改为结点的直接后继。

非空指针呢？仍然指向孩子结点（称为“正常情况”）

例：带了两个标志的某先序遍历结果如下表所示，请画出对应的二叉树。

Ltag	0	0	1	1	1	1	0	1	0	1
data	A	G	E	I	D	J	H	C	F	B
Rtag	0	0	0	1	0	1	0	1	1	1



Ltag=1表示前驱线索
Rtag=1表示后继线索

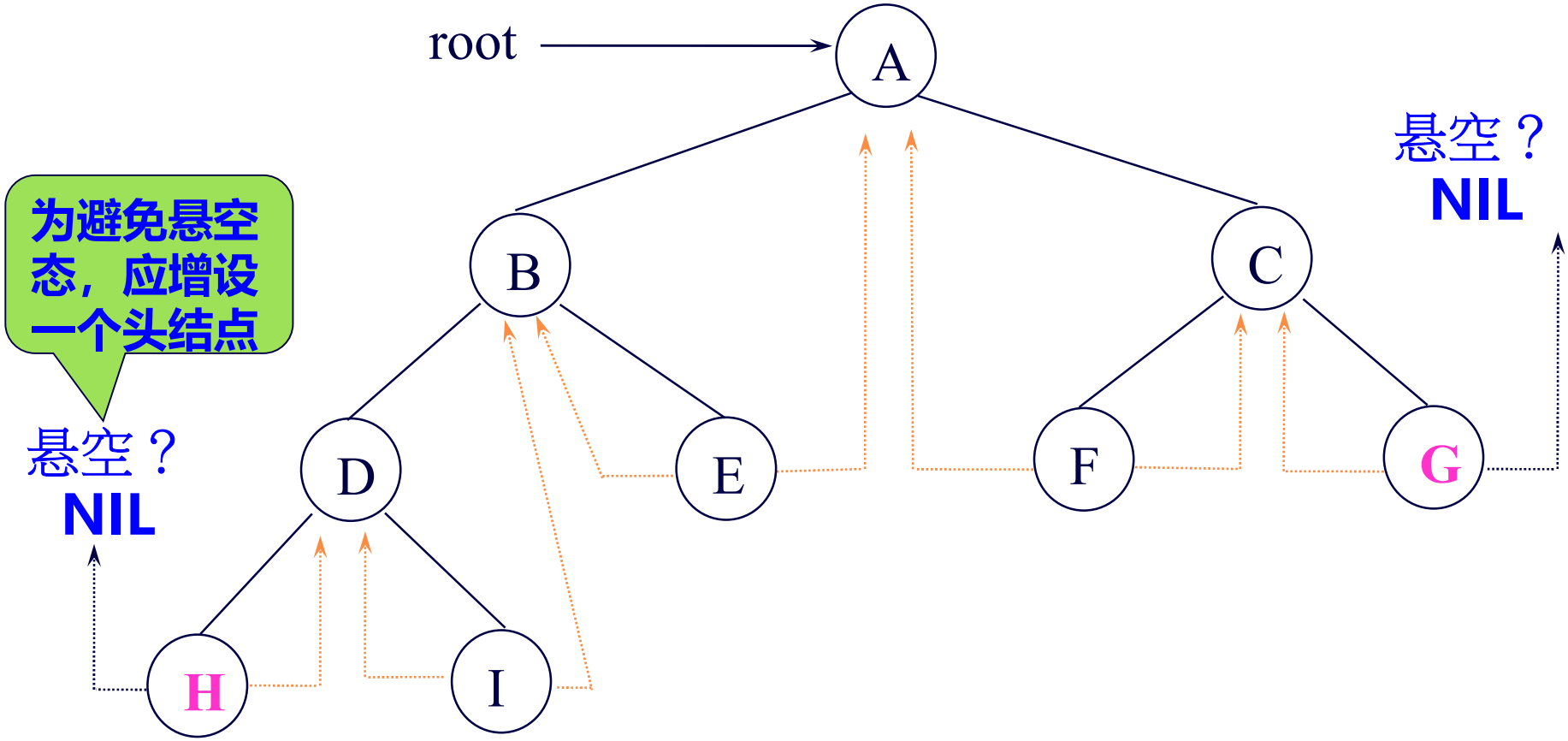
2. 线索二叉树的生成——线索化

线索化过程就是在遍历过程中修改空指针的过程

例1：画出以下二叉树对应的**中序**线索二叉树。

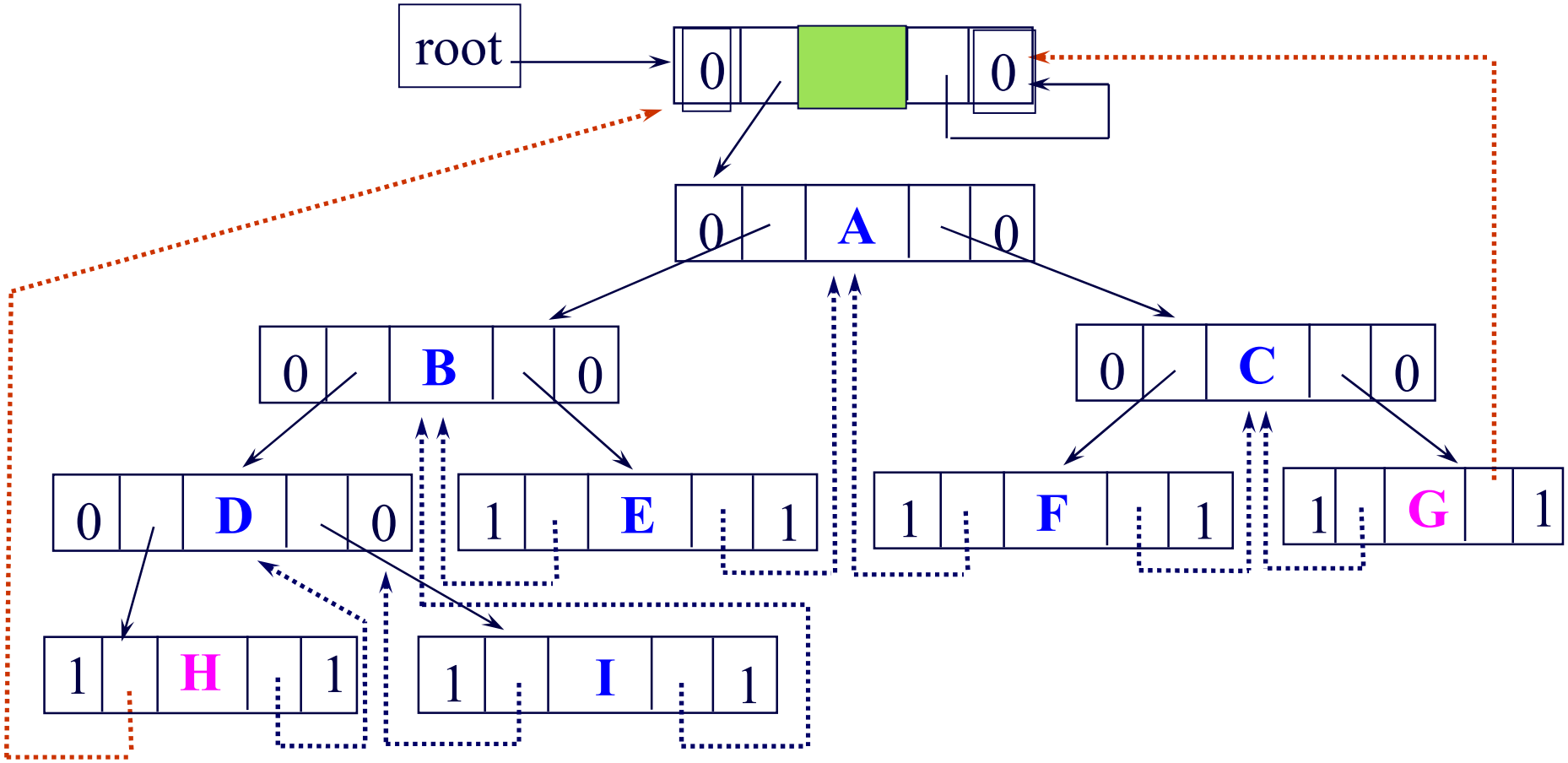
解：对该二叉树**中序**遍历的结果为：H, D, I, B, E, A, F, C, G

所以添加线索应当按如下路径进行：

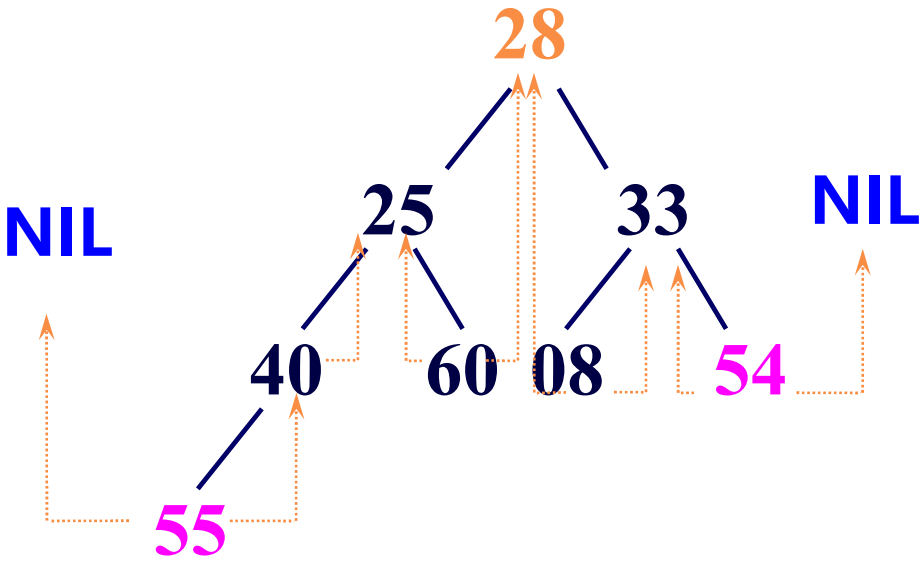


对应的中序线索二叉树存储结构如图所示：

注：此图中序遍历结果为：H, D, I, B, E, A, F, C, G



例2：【 2000年计算机系考研题】给定如图所示二叉树T，请画出与其对应的中序线索二叉树。



解:因为中序遍历序列是：55 40 25 60 28 08 33 54
对应线索树应当按此规律连线，即在原二叉树中添加虚线。

线索二叉树的生成算法（递归算法见教材P134-135）

目的：在遍历二叉树的过程中修改空指针，添加前驱或后继的线索，使之成为线索二叉树。

为了记下遍历过程中访问结点的先后次序，需要设置两个指针：

p指针→当前结点之指针；pre指针→当前结点的前趋结点指针。

设计技巧：依某种顺序遍历二叉树，对每个结点p，判断其左指针是否为空，以及其前驱结点的右指针是否为空。

每次只修改前驱结点的右指针（后继）和本结点的左指针（前驱），参见算法6.6。

若p->lchild = NULL,则{p->Ltag=1;p->lchild = pre;}

//p的前驱线索应存p结点的左边

若pre->rchild = NULL, 则{pre->Rtag = 1;pre->rchild=p;}

//pre的后继线索应存pre结点的右边

3. 线索二叉树的遍历（无需堆栈）

对于线索二叉树的遍历，只要找到序列中的第一个结点，然后依次访问结点的后继直到后继为空为止。
（因为建立线索时已遍历一次，相当于线性化了！）

难点：在线索化二叉树中，并不是每个结点都能直接找到其后继的，当标志为0时，则需要通过一定运算才能找到它的后继。

以中序线索二叉树为例：
当RTag=1时，直接后继指针就在其rchild域内；
当RTag=0时，直接后继是当前结点右子树最左下方的结点；

请注意中序遍历规则是LDR，先左再根再右

附：中序线索二叉树遍历步骤 （算法6.5）：

有后继找后继，
无后继找右子树
的最左子孙

1）设置一个搜索指针p；

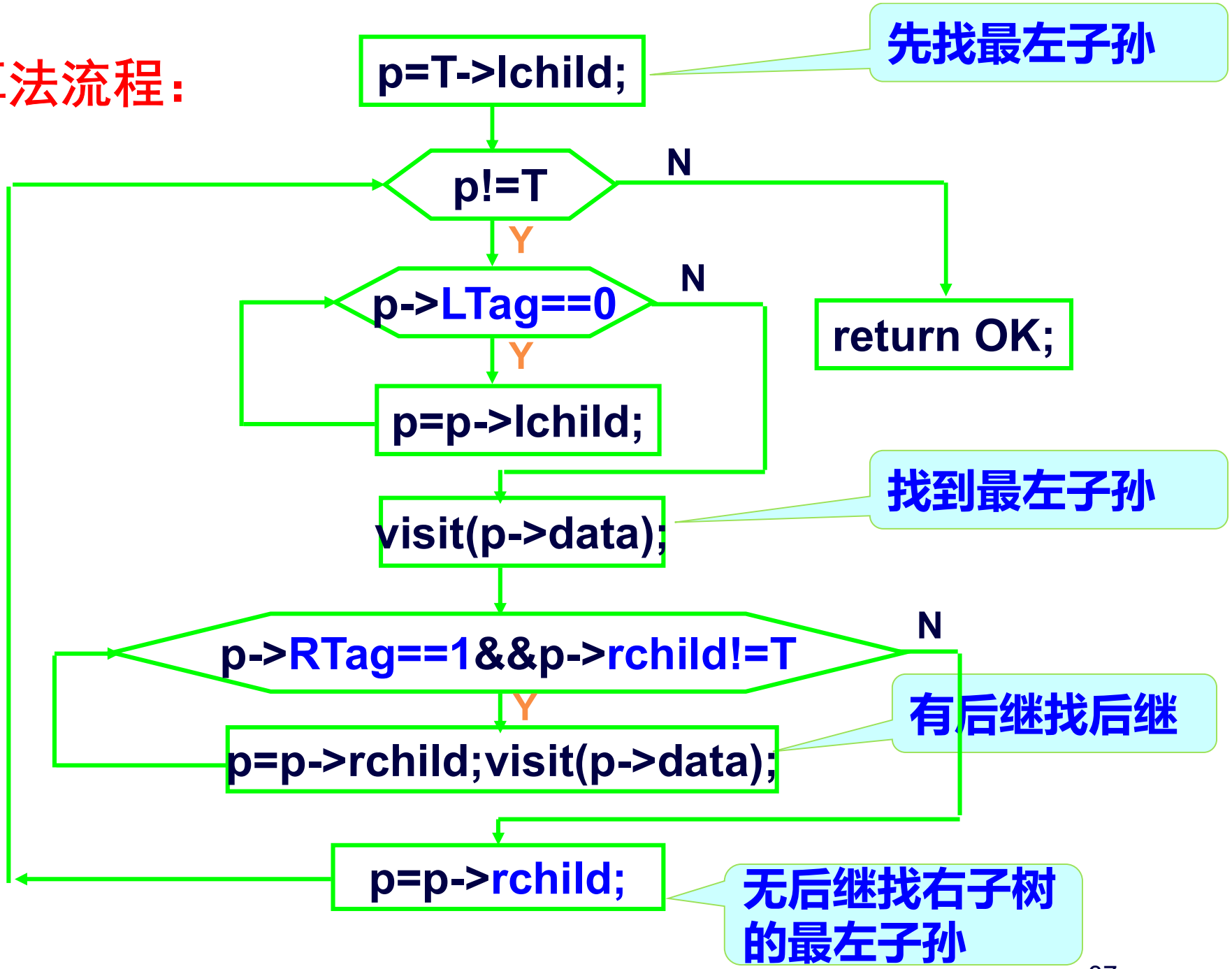
2）先寻找中序遍历之首结点（即最左下角结点），方法是：当LTag=0时（表示有左孩子），p=p->lchild; 直到LTag=1（无左孩子，已到最左下角）；首先访问p->data;

3）接着进入该结点的右子树，检查RTag 和p->rchild；

4） 若该结点的RTag=1(表示有后继线索)，则p=p->rchild；访问p->data ;并重复4），直到后继结点的RTag=0；

5）当RTag=0时（表示有右孩子），此时应当从该结点的右孩子开始(p=p->rchild）查找左下角的子孙结点；即重复2)

算法流程：


p=T->lchild;
先找最左子孙
p!=T
N
Y
return OK;
p->LTag==0
N
Y
p=p->lchild;
visit(p->data);
找到最左子孙
p->RTag==1&&p->rchild!=T
N
Y
p=p->rchild;visit(p->data);
有后继找后继
p=p->rchild;
无后继找右子树的最左子孙